사내 AI 비서

환각부터 평가까지, RAG의 모든 것

FastAPI + LangChain + ChromaDB로 만드는 사내 AI 비서. 환각 체험에서 시작해 검색 품질 평가까지, 하나의 프로젝트로 RAG 전체 여정을 경험합니다.

RAG 실전 가이드

목차

Ch.1: 환각과 RAG의 첫 만남 3

1.1 입사 3일 차, 첫 번째 임무 3

1.2 “당연히 알고 있죠” — 자신감 넘치는 거짓말 3

1.3 “입사 첫날부터 회사 내부를 아는 직원은 없다” 4

1.4 문서를 직접 넣어주면 되지 않을까? 4

1.5 RAG — “오픈북 시험”으로 바꾸기 5

2.1 용어 정리 7

2.2 이번 챕터 파일 구조 7

2.3 실습 환경 구축 7

2.4 실습 순서 8

2.5 실습 1 — step1_fail.py: LLM에게 직접 물어보기 8

2.6 실습 2 — step2_context.py: 문서를 직접 넣어보기 9

2.7 실습 3 — step3_rag.py: RAG 파이프라인 구성 10

2.8 실습 4 — step3_rag_no_chunking.py: 청킹이 왜 필요한가 13

2.9 실습 5 — step4_rag.py: 추론이 필요한 질문 15

2.10 이것만은 기억하세요 17

Ch.1: 환각과 RAG의 첫 만남

이번 버전: 없음 → ex01
한 줄 요약: LLM은 우리 회사 문서를 읽은 적이 없다. 문서를 직접 넣어줘야 한다.
핵심 개념: LLM 환각, Context Injection, RAG

1.1 입사 3일 차, 첫 번째 임무

입사 3일 차, 첫 번째 미션

챕터 오프닝

커넥트에 입사한 지 3일 차. 아직 사내 Wi-Fi 비밀번호를 포스트잇에 적어 모니터에 붙여놓던 시절입니다. 오전 10시, 팀장이 저를 바라보며 다가옵니다.

팀장: "AI로 사내 문서 검색 시스템 만들어봐. 직원들이 규정이나 정책 찾는 게 번거롭다고 해서. 채팅창에 물어보면 바로 답해주는 거."

AI 비서. 사내 문서. 대화식 검색. 나 혼자서?

나: "언제까지요?"

팀장: "급하진 않아. 2주 내로 간단한 프로토타입만."

노트북을 열고 ChatGPT를 실행했습니다.

ChatGPT도 뭐든 대답하잖아. LLM에게 직접 물어보면 되는 거 아니야?

코드부터 짰습니다.

1.2 "당연히 알고 있죠" — 자신감 넘치는 거짓말

연차 규정을 예시로 물어봤습니다.

나: 우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?

LLM이 답했습니다.

> LLM: 커넥트 사의 신입사원 연차 규정은 근로기준법에 따라, 입사 후 1년 미만 기간에는 1개월 개근 시 1일의 유급휴가가 발생합니다. 1년 이상 근무 시에는 15일의 연차가 발생하며, 3년 이상 근무한 경우 1년마다 1일씩 추가됩니다…

그럴듯했습니다. 공식적인 느낌도 나고요. 입사할 때 받은 규정집을 꺼내 비교해봤습니다. 커넥트의 실제 규정은 이랬습니다.

신입사원은 입사 후 3년 동안은 연차가 없다. 대신 매월 1회 '리프레시 데이'를 유급으로 제공한다. 3년 근속 시 30일의 연차가 일시에 발생한다.

잠깐, 뭐라고?

다시 읽었습니다. 완전히 다른 내용이었습니다. LLM이 방금 그럴듯한 거짓말을 한 겁니다.

1.3 "입사 첫날부터 회사 내부를 아는 직원은 없다"

여기서 의문이 생깁니다. LLM은 왜 자신 있게 틀린 대답을 했을까요?

LLM을 이렇게 생각해 보겠습니다. 입사 면접을 보러 온 외부인이라고요. 이 외부인은 세상에 공개된 거의 모든 자료를 읽었습니다. 인터넷, 뉴스, 책, 논문까지. 공개된 텍스트라면 뭐든 섭렵했습니다. 그래서 근로기준법은 완벽하게 알고 일반적인 회사 연차 제도도 줄줄 외웁니다. 그런데 커넥트의 내부 규정집은 공개된 적이 없습니다. 이 외부인이 읽을 방법이 없었어요.

문제는 이 외부인이 "모른다"고 솔직히 말하지 못한다는 점입니다. 질문을 받으면 자기가 아는 것 중에서 가장 비슷해 보이는 걸 자신감 있게 말합니다. "아마 일반적인 회사라면 이렇겠지"라는 추측인데, 마치 확실히 아는 것처럼 들립니다.

이게 LLM 환각(Hallucination) 입니다.

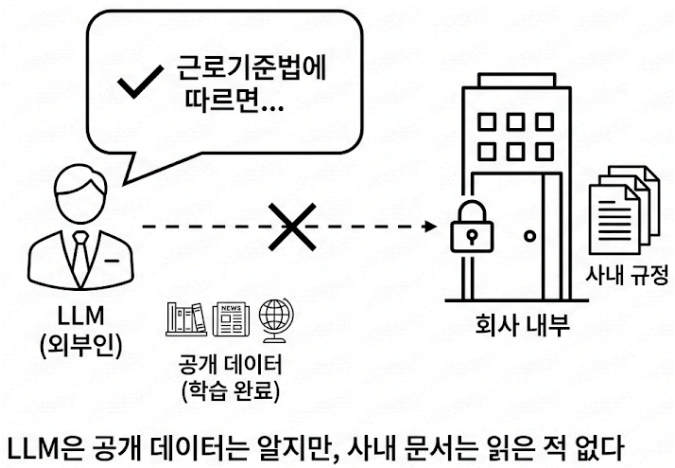


그림 1-1: LLM은 세상의 공개 데이터는 학습했지만, 우리 회사 내부 문서는 읽은 적이 없다.

GPT든 Claude든 Gemini든 마찬가지입니다. 학습 데이터에 없는 정보는 알 방법이 없어요. 그런데 솔직히 "모른다"고 하지 않고 그럴듯하게 지어냅니다. 일반적인 내용과 비슷한 맥락일수록 더 자연스럽게 지어내고요. 커넥트의 연차 규정은 공개된 인터넷 어디에도 없습니다. LLM이 알 리가 없죠. 근로기준법 기반으로 그럴듯한 답을 만들어낸 것뿐입니다.

1.4 문서를 직접 넣어주면 되지 않을까?

생각해보면 해결책은 단순합니다. LLM이 모른다면 직접 알려주면 되지 않을까요?

규정 내용을 통째로 프롬프트에 붙여서 다시 물어봤습니다.

> **나**: 아래 [커넥트 취업규칙]을 참고해서 신입사원 연차 규정을 알려줘.

이번엔 달랐습니다. 커넥트의 실제 규정을 정확히 설명해줬습니다.

(오, 이거면 되는 거 아니야?)

그런데 사내 문서가 규정집 하나가 아닙니다. 복지 정책, 보안 지침, 업무 가이드, 회의록, 프로젝트 문서까지 파일만 수십 개입니다. 매번 전부 복사해서 프롬프트에 붙이면 어떻게 될까요? LLM에는 한 번에 처리할 수 있는 텍스트 길이 한도가 있습니다. 문서가 쌓일수록 한도를 넘기기 쉽고요. 무엇보다 연차 규정을 물어보는데 보안 지침이나 복지 정책까지 다 넣어서 보내는 건 비효율적입니다. 관련 없는 내용이 섞일수록 LLM이 정작 필요한 부분을 놓치기 쉬워집니다.

문서를 통째로 넣는 방식은 임시방편이었습니다.

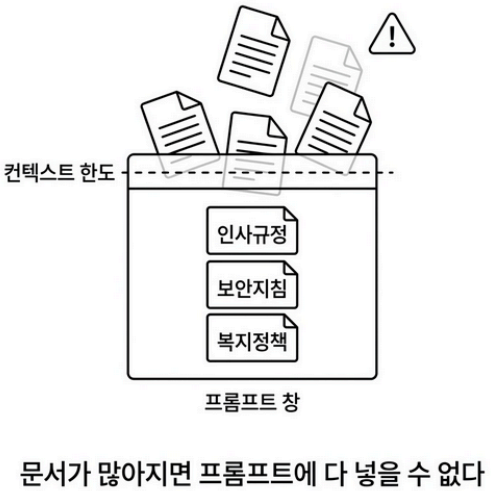


그림 1-2: 문서를 통째로 넣는 방식의 한계. 문서가 늘어나면 프롬프트 창이 넘친다.

1.5 RAG — "오픈북 시험"으로 바꾸기

더 나은 방법이 있습니다.

LLM이 모든 사내 문서를 외울 필요가 있을까요? 사람도 비슷한 문제를 해결한 방식이 있습니다. 시험에서 모든 내용을 통째로 외우는 대신 오픈북을 허용하면 됩니다. 시험지가 나오면 그 문제와 관련된 페이지를 찾아서 보면서 답하는 거죠.

LLM도 마찬가지입니다. 사내 문서 전체를 외울 필요가 없어요. 질문이 들어왔을 때 **그 질문과 관련된 문서 조각만 찾아서 LLM에게 건네주면 됩니다.**

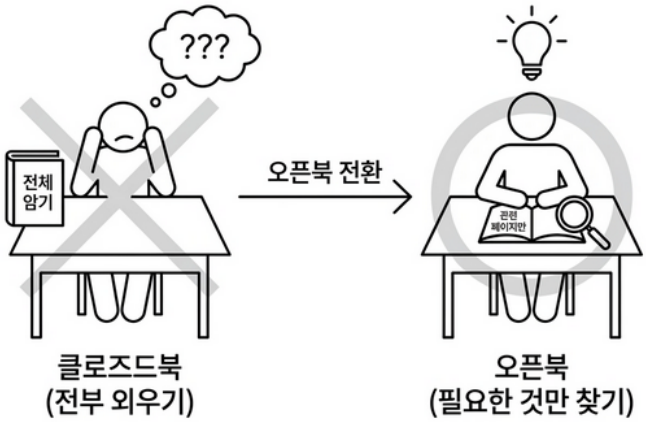
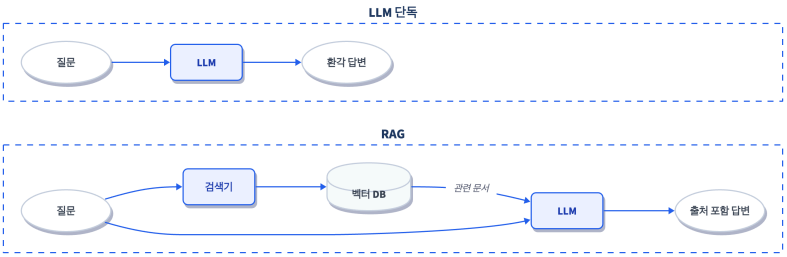


그림 1-3: 클로즈드북 vs 오픈북. RAG는 LLM에게 오픈북 시험을 치르게 하는 것이다.

이것이 **RAG** — Retrieval-Augmented Generation, 검색 증강 생성입니다.

흐름을 보면 이렇게 됩니다.



LLM 단독 호출과 RAG의 차이

그림 1-4: LLM 단독 호출과 RAG의 차이. RAG는 질문마다 관련 문서를 찾아서 LLM에 건네준다.

1. 사내 문서들을 미리 **벡터 DB**에 조각으로 나눠 저장해 놓습니다 (오픈북 준비)
2. 질문이 들어오면, 그 질문과 의미가 비슷한 문서 조각을 벡터 DB에서 찾습니다 (관련 페이지 찾기)
3. 찾은 문서 조각 + 질문을 LLM에게 함께 넘깁니다
4. LLM이 그 문서를 보면서 답합니다 (오픈북으로 시험 보기)

이제 LLM이 우리 회사 규정을 외울 필요가 없습니다. 질문할 때마다 관련 규정을 찾아서 보여주면 되니까요. 어느 문서를 참고했는지도 함께 돌려줄 수 있고요. 이번 챕터의 목표는 이 흐름을 직접 만들어보는 겁니다. 더미 문서 3개짜리 간단한 버전으로 시작하겠습니다. 실제 PDF 파싱이나 한국어 임베딩 모델 적용, DB 연동은 뒤 챕터에서 차례로 붙입니다.

이제 실습으로 LLM 환각을 직접 확인하고, RAG로 해결해보겠습니다.

2.1 용어 정리

이야기 속 표현	진짜 용어	정식 정의
“자신감 넘치는 거짓말”	LLM 환각 (Hallucination)	LLM이 학습 데이터에 없는 정보를 그럴듯하게 만들어내는 현상
“문서를 직접 붙여 넣기”	Context Injection	관련 정보를 프롬프트에 직접 넣어서 LLM에 제공하는 방법
“오픈북 시험”	RAG (Retrieval-Augmented Generation)	외부 지식 저장소에서 관련 문서를 검색해 LLM 생성에 활용하는 방식
“오픈북 준비”	임베딩 + 벡터 DB 저장	문서를 수치 벡터로 변환해 ChromaDB에 인덱싱하는 과정
“관련 페이지 찾기”	벡터 유사도 검색	질문 벡터와 문서 벡터 간 코사인 유사도를 계산해 가장 관련 있는 문서를 반환

2.2 이번 챕터 파일 구조

```
ex01/
├── step1_fail.py            [실습] LLM 단독 호출 → 환각 체험
├── step2_context.py         [실습] 컨텍스트 직접 주입 → 임시 해결
├── step3_rag.py             [실습] RAG 기본 파이프라인 구성
├── step3_rag_no_chunking.py [실습] 청킹 없이 비교 → 차이 체감
└── step4_rag.py             [실습] 추론 심화 (Chain-of-Thought)
```

이 챕터는 더미 문서(3개)로 동작을 확인하는 맛보기 버전입니다. 실제 PDF/DOCX 파싱과 영속 저장은 CH04(VectorDB 구축)에서 다룹니다.

2.3 실습 환경 구축

기본 환경(Python 3.12, Ollama)이 아직 없다면 **부록(환경 설정)** 을 먼저 참고하세요.

```
cd ex01
python3.12 -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Ollama 모델이 아직 없다면 다운로드합니다.

```
ollama pull deepseek-r1:8b
ollama pull nomic-embed-text
```

팁: LLM 선택 기본값은 Ollama + `deepseek-r1:8b`입니다(16GB RAM 이상 권장). RAM이 부족하거나 응답이 너무 느리면 `.env`에서 `LLM_PROVIDER=openai`로 바꿔서 GPT-4o-mini를 쓸 수도 있습니다. 단, API 비용이 발생합니다. `.env` 파일에 `OPENAI_API_KEY=sk-xxxxxx` 형태로 키를 등록하세요. 상세 안내는 **프롤로그** 의 "시작하기 전에"를 참고하세요.

이번 챕터에서는 LangChain 이라는 프레임워크를 사용합니다. LLM 호출, 벡터 검색, 체인 조립처럼 RAG에 필요한 부품을 제공하는 도구입니다. 여기서는 맛보기로만 쓰고 CH05에서 본격적으로 다룹니다.

패키지	역할
`langchain-ollama`	Ollama LLM/임베딩 연동
`langchain-chroma`	ChromaDB 벡터 저장소
`langchain-classic`	RetrievalQA 체인 (CH05에서 LCEL로 전환)
`chromadb`	벡터 DB

팁: 지금은 개념만 잡으세요 LangChain, 임베딩, 벡터 DB 같은 용어가 한꺼번에 나와서 부담스러울 수 있습니다. 지금은 "문서를 넣어주면 LLM이 정확하게 답한다"는 **RAG의 개념** 만 잡으면 충분합니다. 각 기술의 동작 원리는 CH04~CH05에서 차근차근 다룹니다.

2.4 실습 순서

그림 1-5: 실습 순서. step1부터 순서대로 실행한다.

환각을 직접 체험하고(step1), 문서를 넣으면 달라지는 걸 확인한 뒤(step2), RAG로 조립합니다(step3). 그다음 청킹 없이 돌려서 차이를 체감하고(step3_no_chunking), 추론이 필요한 질문까지 던져봅니다(step4). **step1부터 순서대로 실행하세요.**

2.5 실습 1 — step1_fail.py: LLM에게 직접 물어보기

아래 코드를 `ex01/step1_fail.py`에 작성합니다.

```
from langchain_ollama import ChatOllama

# 로컬 LLM 연결
llm = ChatOllama(model="deepseek-r1:8b", temperature=0)
```

```
# 질문: 모델이 학습했을 리 없는 가상의 회사 규정
question = "우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?"

print(f"질문: {question}\n")
response = llm.invoke(question)
print(f"답변:\n{response.content}")
```

ChatOllama는 LangChain이 Ollama LLM을 호출할 때 쓰는 래퍼입니다. **temperature=0**은 LLM이 창의적 변형 없이 가장 확률 높은 답변을 내놓게 하는 설정이에요. 실행하면 그럴듯하게 들리지만 커넥트의 실제 규정과는 다른 답변이 나옵니다.

```
# 실행
python step1_fail.py
```

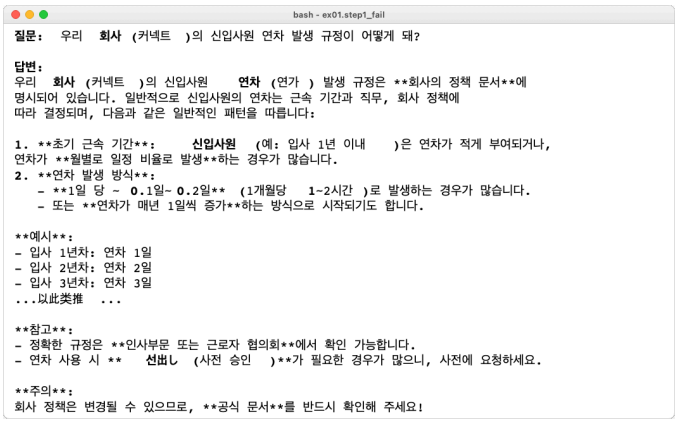


그림 1-5: step1_fail.py 실행 결과. 자신감 있게 답하지만 실제 커넥트 규정과 다르다.

2.6 실습 2 — step2_context.py: 문서를 직접 넣어보기

step1에서 LLM이 거짓말하는 걸 봤습니다. 이번에는 **규정 내용을 프롬프트에 직접 포함** 시켜 봅니다. 아래 코드를 **ex01/step2_context.py**에 작성합니다.

```
from langchain_ollama import ChatOllama

llm = ChatOllama(model="deepseek-r1:8b", temperature=0)

# 1. 정보를 변수에 담습니다 (아직 DB 안 씀)
context_data = """
[커넥트 취업규칙]
1. 신입사원은 입사 후 3년 동안은 연차가 없다. (파격적인 규정)
2. 대신 매월 1회 '리프레시 데이'를 유급으로 제공한다.
```

```
3. 3년 근속 시 30일의 연차가 일시에 발생한다.
"""

question = "우리 회사(커넥트)의 신입사원 연차 발생 규정이 어떻게 돼?"

# 2. 프롬프트에 정보를 포함시킵니다
prompt = f"""
아래 [참고 정보]를 보고 질문에 답해줘.
[참고 정보]
{context_data}

질문: {question}
"""

print(f"질문: {question}\n")
response = llm.invoke(prompt)
print(f"답변:\n{response.content}")
```

step1과 달라진 부분은 `context_data`를 프롬프트에 직접 넣었다는 것뿐입니다. 이제 정확한 답변이 나옵니다. 하지만 한계도 바로 보여요. 문서 하나면 괜찮지만 수십 개를 매번 통째로 붙이면 프롬프트가 엄청나게 길어집니다. LLM이 처리할 수 있는 텍스트 길이에는 한도(컨텍스트 윈도우)가 있으니까요.

```
# 실행
python step2_context.py
```

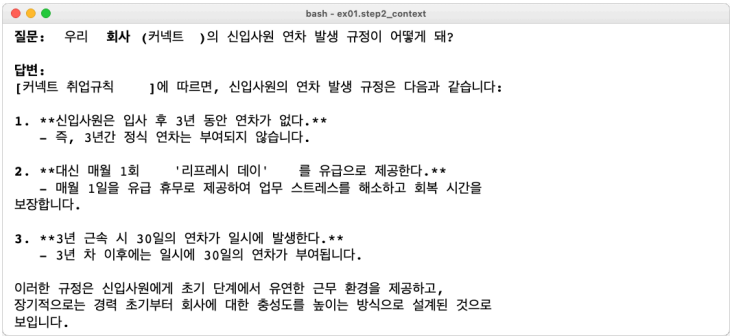


그림 1-6: step2_context.py 실행 결과. 문서를 직접 넣으니 정확하게 답한다.

2.7 실습 3 — step3_rag.py: RAG 파이프라인 구성

step2에서는 문서를 수동으로 넣었습니다. 이번에는 **벡터 DB에 문서를 저장하고 질문에 맞는 문서를 자동으로 찾아오는** RAG 파이프라인을 만들어 봅니다. 아래 코드를 `ex01/step3_rag.py`에 작성합니다.

```python
from langchain_classic.chains import RetrievalQA
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate

# 1. 더미 데이터 준비 — 문서 3개를 Document 객체로 만듭니다
docs = [
    Document(
        page_content="[인사규정] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은 "
        "법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이'를 "
        "휴가로 사용할 수 있습니다.",
        metadata={"source": "인사규정"},
    ),
    Document(
        page_content="[보안규정] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안 USB만 "
        "사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.",
        metadata={"source": "보안규정"},
    ),
    Document(
        page_content="[복지규정] 식대 지원: 점심 식사는 무제한 법인카드로 지원하며, "
        "저녁 식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.",
        metadata={"source": "복지규정"},
    ),
]

# 2. VectorDB 생성 — 문서를 임베딩하여 ChromaDB에 저장
print("문서를 학습(임베딩) 중입니다...")
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(documents=docs, embedding=embeddings)

# 3. 검색기 + LLM 체인 연결
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

template = """당신은 회사의 규정에 대해 설명해주는 AI 비서입니다.
아래의 참고 정보를 바탕으로 질문에 답하세요. 반드시 한국어로 답변해야 합니다.

참고 정보: {context}

질문: {question}
답변:"""

llm = ChatOllama(model="deepseek-r1:8b", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
```

```
    chain_type_kwargs={
        "prompt": PromptTemplate(template=template, input_variables=["context", "question"])
    },
)

# 4. 질문하고 출처 확인
question = "신입사원 휴가 규정에 대해 알려줘."
print(f"\n질문: {question}")
print("-" * 30)

result = qa_chain.invoke({"query": question})

print("\n--- 검색된 문서(근거) ---")
for doc in result["source_documents"]:
    print(f"[{doc.metadata['source']}]: {doc.page_content}")

print("\n--- AI 답변 ---")
print(result["result"])
```

코드가 길어 보이지만 흐름은 네 단계입니다.

1. **문서 준비** — `Document` 객체 3개를 만듭니다. 인사규정, 보안규정, 복지규정이에요.
2. **벡터 DB 저장** — `OllamaEmbeddings`가 각 문서를 수백 차원의 숫자 배열(벡터)로 변환합니다. `Chroma.from_documents()`가 이 벡터를 ChromaDB에 저장하고요.
3. **검색기 + LLM 연결** — `k=3`은 "질문과 가장 비슷한 문서 3개를 가져오라"는 설정입니다. `RetrievalQA`가 검색기와 LLM을 체인으로 연결합니다.
4. **질문 + 출처 확인** — `return_source_documents=True` 덕분에 어떤 문서를 참고했는지도 함께 돌아옵니다.

> **이 챕터의 임베딩 모델**: `nomic-embed-text`를 사용합니다. CH04에서 한국어에 최적화된 `ko-sroberta-multitask`로 교체합니다.

```
# 실행
python step3_rag.py
```

```
bash - ex01.step3_rag
문서를   학습 (임베딩 ) 중입니다   ...

질문: 신입사원 휴가 규정에 대해 알려줘.
--------------------------------

--- 검색된   문서 (근거 ) ---
[복지규정   ]: [복지규정   ] 식대 지원: 점심 식사는 무제한 법인카드로 지원하며, 저녁
식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.
[보안규정   ]: [보안규정   ] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안
USB만 사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.
[인사규정   ]: [인사규정   ] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은
법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급              '리프레시 데이'    를 휴가로
사용할 수 있습니다.

--- AI 답변 ---
신입사원 휴가 규정에 대해 알려드리겠습니다.

신입사원은 입사 후 처음 3년 동안 법정 연차가 발생하지 않습니다. 대신 매월 1회의
유급   '리프레시 데이'    를 휴가로 사용할 수 있습니다. 이는 회사에서 제공하는 특별
휴가로, 사용 시에는 일반 연차와 마찬가지로 업무 일정 조정이 필요합니다.

추가적인 휴가 사용에 관한 세부 사항은 인사부문과 상담하시는 것이 좋습니다.
```

그림 1-7: step3_rag.py 실행 결과. [인사규정] 문서를 찾아서 답변하고, 어디서 가져왔는지 출처까지 보여준다.

이제 답변과 함께 어느 문서를 참고했는지가 나옵니다. step2에서는 문서를 수동으로 넣어줬지만 이번에는 **질문에 맞는 문서를 자동으로 찾아왔습니다.** 환각이 사라지고 출처가 생겼습니다.

2.8 실습 4 — step3_rag_no_chunking.py: 청킹이 왜 필요한가

step3에서 문서 3개를 **각각 따로** 벡터 DB에 저장했습니다. 이번에는 반대로, 모든 문서를 **하나의 덩어리로 합쳐서** 저장하면 어떻게 되는지 비교해 봅니다. 아래 코드를 `ex01/step3_rag_no_chunking.py`에 작성합니다.

```python
from langchain_classic.chains import RetrievalQA
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate

# 1. 청킹 미적용: 모든 텍스트를 하나의 문자열로 합침 (통짜 데이터)
context_all = """
[인사규정] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은 법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데
이'를 휴가로 사용할 수 있습니다.
[보안규정] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안 USB만 사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.
[복지규정] 식대 지원: 점심 식사는 무제한 법인카드로 지원하며, 저녁 식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.
"""

# 하나의 거대한 문서로 만듦 -> 검색이 비효율적임
docs_bad = [Document(page_content=context_all, metadata={"source": "전체규정"})]

# 2. VectorDB 생성
print("문서를 학습(임베딩) 중입니다... (청킹 미적용)")
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(documents=docs_bad, embedding=embeddings)

# 3. 검색기 및 프롬프트 설정 (통째로 하나뿐이므로 k=1로 검색해도 전체가 다 나옴)
```

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 1})

template = """당신은 회사의 규정에 대해 설명해주는 AI 비서입니다.
아래의 참고 정보를 바탕으로 질문에 답하세요. 반드시 한국어로 답변해야 합니다.

참고 정보: {context}

질문: {question}
답변:"""
PROMPT = PromptTemplate(template=template, input_variables=["context", "question"])

# 4. RAG 체인 실행
llm = ChatOllama(model="deepseek-r1:8b", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type_kwargs={"prompt": PROMPT},
    return_source_documents=True,
)

question = "신입사원 휴가 규정에 대해 알려줘."
print(f"\n질문: {question}")
print("-" * 30)

result = qa_chain.invoke({"query": question})
print(f"\nAI 답변:\n{result['result']}")
```

step3과 비교하면 달라진 부분은 딱 하나입니다. 문서 3개를 **하나의 문자열** (`context_all`)로 합쳐서 `Document` 1개로 만들었어요. 벡터 DB에 저장되는 문서가 하나뿐이므로 `k=1`로도 전체가 다 나옵니다.

두 파일을 직접 실행해서 결과를 비교해 보세요.

```
# 실행
python step3_rag_no_chunking.py
```

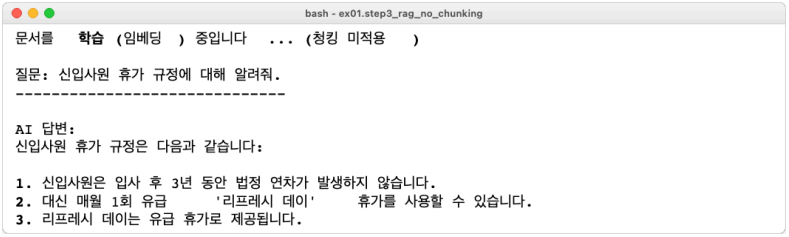


그림 1-8: 청킹 여부에 따른 검색 결과 비교. 조각으로 나누면 관련 문서만 정확히 찾는다.

step3에서는 인사규정만 깔끔하게 찾아왔지만 여기서는 인사규정 + 보안규정 + 복지규정이 통째로 들어옵니다. 관련 없는 내용이 섞이면 LLM이 정작 필요한 부분을 놓치기 쉽습니다. 문서를 조각으로 나누는 것, 즉 **청킹(Chunking)** 이 왜 필요한지 체감되실 겁니다. 청킹 전략의 상세 비교는 CH08(검색 품질 튜닝)에서 다룹니다.

2.9 실습 5 — step4_rag.py: 추론이 필요한 질문

step3까지의 질문은 "규정이 뭐야?" 같은 단순 검색이었습니다. 이번에는 **규정을 찾아서 읽고 계산까지 해야 하는 질문** 을 던져봅니다. 아래 코드를 `ex01/step4_rag.py`에 작성합니다.

```
from langchain_classic.chains import RetrievalQA
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings, ChatOllama
from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate

# 1. 더미 데이터 준비
docs = [
    Document(page_content="[인사규정] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은 법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이'를 휴가로 사용할 수 있습니다.", metadata={"source": "인사규정"}),
    Document(page_content="[보안규정] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안 USB만 사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.", metadata={"source": "보안규정"}),
    Document(page_content="[복지규정] 식대 지원: 점심 식사는 무제한 법인카드로 지원하며, 저녁 식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.", metadata={"source": "복지규정"}),
]

# 2. VectorDB 생성
print("문서를 학습(임베딩) 중입니다...")
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(documents=docs, embedding=embeddings)

# 3. 검색기(Retriever) 설정
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

# 4. 프롬프트 템플릿
template = """당신은 회사의 규정에 대해 설명해주는 AI 비서입니다.
아래의 참고 정보를 바탕으로 질문에 답하세요. 반드시 한국어로 답변해야 합니다.

참고 정보: {context}

질문: {question}
답변:"""

PROMPT = PromptTemplate(
    template=template, input_variables=["context", "question"]
)
```

```python
# 5. RAG 체인 연결
llm = ChatOllama(model="deepseek-r1:8b", temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True,
    chain_type_kwargs={"prompt": PROMPT},
)

# 6. 질문하기 — 추론이 필요한 복잡한 질문
question = "입사 6개월차 신입인데 리프레시 데이 2번 썼어. 몇 번 남았는지 규정 기반으로 계산해줘."
print(f"\n질문: {question}")
print("-" * 30)

result = qa_chain.invoke({"query": question})

print("\n--- 검색된 문서(근거) ---")
for doc in result["source_documents"]:
    print(f"[{doc.metadata['source']}]: {doc.page_content}")

print("\n--- AI 답변 ---")
print(result["result"])
```

코드 구조는 step3과 거의 같습니다. 달라진 건 **질문** 뿐이에요. "매월 1회 제공" → "6개월이면 6번" → "2번 썼으면 4번 남음"까지, 규정을 읽고 계산해야 하는 질문입니다.

```bash
# 실행
python step4_rag.py
```

```
bash - ex01.step4_rag
문서를  학습 (임베딩 ) 중입니다  ...

질문: 입사 6개월차 신입인데 리프레시 데이 2번 썼어. 몇 번 남았는지 규정 기반으로
계산해줘.
------------------------------

--- 검색된  문서 (근거 ) ---
[보안규정  ]: [보안규정  ] 업무 보안: 모든 임직원은 회사에서 지급한 승인된 보안
USB만 사용해야 하며, 개인 USB나 외부 저장 매체 사용은 엄격히 금지됩니다.
[복지규정  ]: [복지규정  ] 식대 지원: 점심 식사는 무제한 법인카드로 지원하며, 저녁
식사는 오후 9시 이후 야근 시에만 사용이 가능합니다.
[인사규정  ]: [인사규정  ] 신입사원 휴가 및 연차: 신입사원은 입사 후 처음 3년 동안은
법정 연차가 발생하지 않습니다. 대신 매월 1회의 유급 '리프레시 데이' 를 휴가로
사용할 수 있습니다.

--- AI 답변 ---
입사 6개월차 신입사원으로서 총 6개월 동안 매월 1회씩 리프레시 데이를 사용할 수
있습니다.
이미 2회 사용했으므로, 남은 리프레시 데이는 **6 - 2 = 4회**입니다.

**참고**: 신입사원은 입사 후 3년 동안 매월 1회씩 리프레시 데이를 사용할 수
있습니다.
```

실행 결과 1-9: step4_rag.py 실행 결과. 규정을 바탕으로 연차를 스스로 계산하고 추론해 낸 모습이다.

DeepSeek R1은 `<think>` 태그 안에서 단계별로 생각하는 **Chain-of-Thought** 추론을 합니다. 실행하면 검색된 문서(근거)와 함께 계산 과정이 포함된 답변이 나옵니다.

2.10 이것만은 기억하세요

- **LLM은 우리 회사 문서를 읽은 적이 없습니다.** 아무리 자신감 있게 답해도 사내 정보는 우리가 직접 넣어줘야 합니다.
- **RAG는 오픈북 시험입니다.** LLM이 모든 걸 외울 필요 없이 질문마다 관련 문서를 찾아보면서 답합니다.
- 이 챕터의 `RetrievalQA`는 구버전 API입니다. CH05에서 LCEL 파이프라인으로 교체하고, CH04에서 ChromaDB를 디스크에 영구 저장하는 방식으로 바꿉니다.
- 다음 챕터에서는 AI 비서가 조회할 실제 사내 시스템(직원, 연차, 매출 DB)을 FastAPI로 만들어 봅니다.